SINE WAVE GENERATOR

Daniel de Lizaur García

The aim of this project is to construct a circuit capable of generating a sine wave signal. This device could have a multitude of applications, for example inside of lab equipment (oscilloscope) or even an electronic musical instrument. Something to note is that it is possible to generate a sine wave using exclusively analog components, however, because this course is dedicated to digital electronics and microcontrollers, we will incorporate a microcontroller into the design.

The first solution I considered was generating the sine wave in software, using the microcontroller. Then, those digital "samples" would be sent to a Digital-to-Analog Converter, which would output the desired analog sine wave signal. There are several ways to generate the sine wave, for example:

- Using some infinite series approximation such as the Taylor series. This approach is memory efficient, but computationally expensive if we want good accuracy.
- Using a lookup table. This approach is a lot more efficient and more commonly used, but requires a lot of memory to store the table. Achieving good accuracy may require the use of external ROM chips, which would needlessly increase the complexity of the project.

Later on, I found out about DDS (Direct Digital Synthesis) chips. This kind of chip interfaces with the microcontroller using some sort of protocol (most commonly serial). We can then send commands containing the frequency, amplitude and phase, and the DDS chip will generate the desired signal.
With this in mind, I designed the device in the following steps:

1. Selecting a microcontroller.
2. Selecting a DDS chip.
3. Sending commands to the DDS chip and generating a signal.
4. Adding user input to change the frequency (Matrix keyboard, USB)
5. Adding an LCD screen to visualize the current frequency.

MICROCONTROLLER

For the microcontroller choice, I considered the following requirements:

- 8051-based architecture.
- Integrated USB support (this requirement was added later in the process)
- SPI implemented in hardware (to simplify the firmware)

After some research, I found the AT89C5131A-S3SUM, which satisfies all the requirements and also has a low cost (10-15 USD depending on the quantity and supplier)
https://www.digikey.com/en/products/detail/microchip-technology/AT89C5131A-S3SUM/1008478

DDS CHIP (4 pins)

For the DDS chip, I selected the AD9837 chip by Analog Devices (https://www.analog.com/en/products/ad9837.html), which has a very low cost (around 2 USD for large orders) and all has the features required for the project. The frequency, amplitude and phase can be changed by sending commands through a SPI interface, which only requires four pins. This chip accomplishes the same I was trying before: internally, it generates digital samples using a lookup table, which are passed to a DAC.

It is connected to the microcontroller with a MOSI (master output, slave input) line, SCK (serial clock), and FSYNC (similar to chip select/enable). The MISO line of the microcontroller is not used because communication is unidirectional from the microcontroller to the DDS chip.

The SCK frequency can be set with a special register in the microcontroller. With the oscillator configuration I used (using both XAL1 and XAL2 pins), the maximum frequency is 36Mhz. The fastest possible baud rate is 1/4 of the oscillator signal, which is 9Mhz. This same clock signal is directed to the MCLK (main clock) input.

To send a command to the AD9837, the microcontroller must set FSYNC to low. Then, a 16 bit word can be sent serially. Once the word is sent, FSYNC must be set back to high. Some commands require sending two words, such as changing the frequency, because the frequency register is 28 bits wide.

The output of the AD9837 cannot be used directly, so it is passed through a voltage follower, implemented with an operational amplifier. I have also added decoupling capacitors between VCC and GND, and between CAP/2.5V and DGND, as they were required in the datasheet.

MATRIX KEYBOARD (8 pins)

As for the keyboard, I decided that a multiplexed matrix keyboard in a 4x4 configuration would be perfect for this project. It would allow adding keys for the numbers 0 to 9, an enter key, a cancel key, as well as three keys to select whether we want to edit the frequency, amplitude or phase or the wave. This leaves an extra available key, which could be used for extra functionality, such as changing the type of wave (the DDS is capable of producing sine, triangle and pulse waves). Pressing the key would switch the type in a cyclical way.

To implement the keyboard, I have connected the columns to pins P2.0-P2.3 and the rows to P2.4-P2.7. These ports have an internal pullup resistor, so an external resistor is not necessary. We check every combination of row and column and if we detect current, it means that the key corresponding to that row and column was pressed. It may be necessary to introduce debouncing by checking the state every ~20ms. We can use a membrane keypad such as this one (1.50 USD)
https://www.kubii.com/en/controllers-actuators/2478-matrix-keypad-4x4-membrane-3272496014282.html

DISPLAY (8+5 pins)

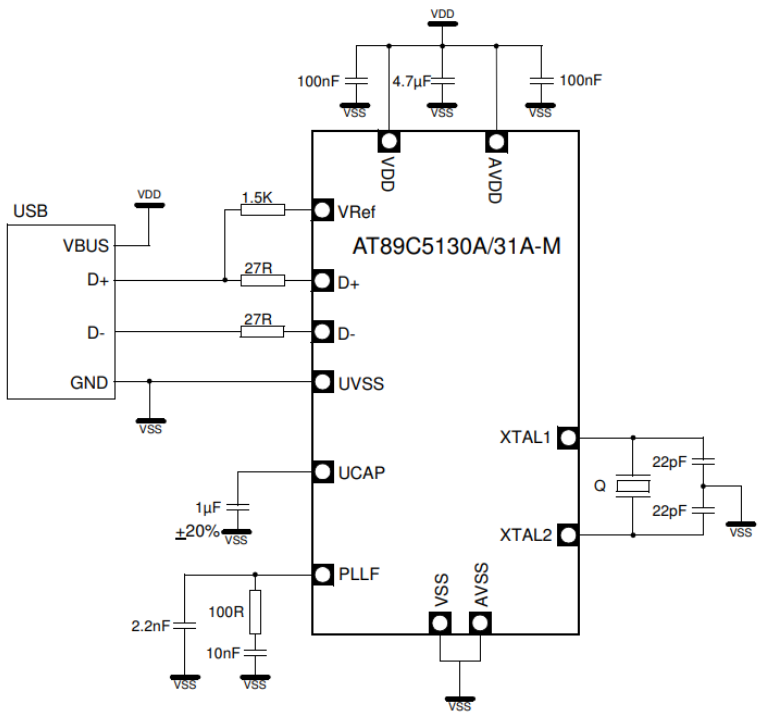
Having a display would be very useful to see the value of the parameter we're editing (frequency, amplitude or phase). Using five 7-segment displays would be appropriate for the range of frequencies we are able to generate, so it is clear that the best option would be to use a dynamic display, otherwise we would run out of available pins.
However, I later decided to use a 16x2 matrix display, because of the extra versatility. For this device I am using the HY1602E, which has a cost of around 3 USD. https://www.ebuy7.com/hy1602e-lcd-screen-16x2-lcd-module-dimensions-84x44x12-lcd-screen.html

The microcontroller communicates with the LCD using P3.0-P3.7, as well as the Enable and R/S signals. The R/W selector is tied to ground because it is not necessary to read from the LCD's memory.

USB INTERFACE

With the current design, changing some parameter of the wave requires entering the value manually using the keyboard. However, if we added a USB interface to the device, we could write a program to change the value automatically from a computer, which opens up many new applications for the project, such as frequency modulation, amplitude modulation, or even a sound synthesizer capable of producing music. First, I considered using a dedicated USB to SPI chip, but later, I found some microcontrollers with integrated USB support.

According to the microcontroller's spreadsheet, this circuitry is required for the chip to work.



This already includes the USB connection with VRef, D+, D- and UVSS and the oscillator circuit. I have also added a reset switch.

PROJECT ASSUMPTIONS:

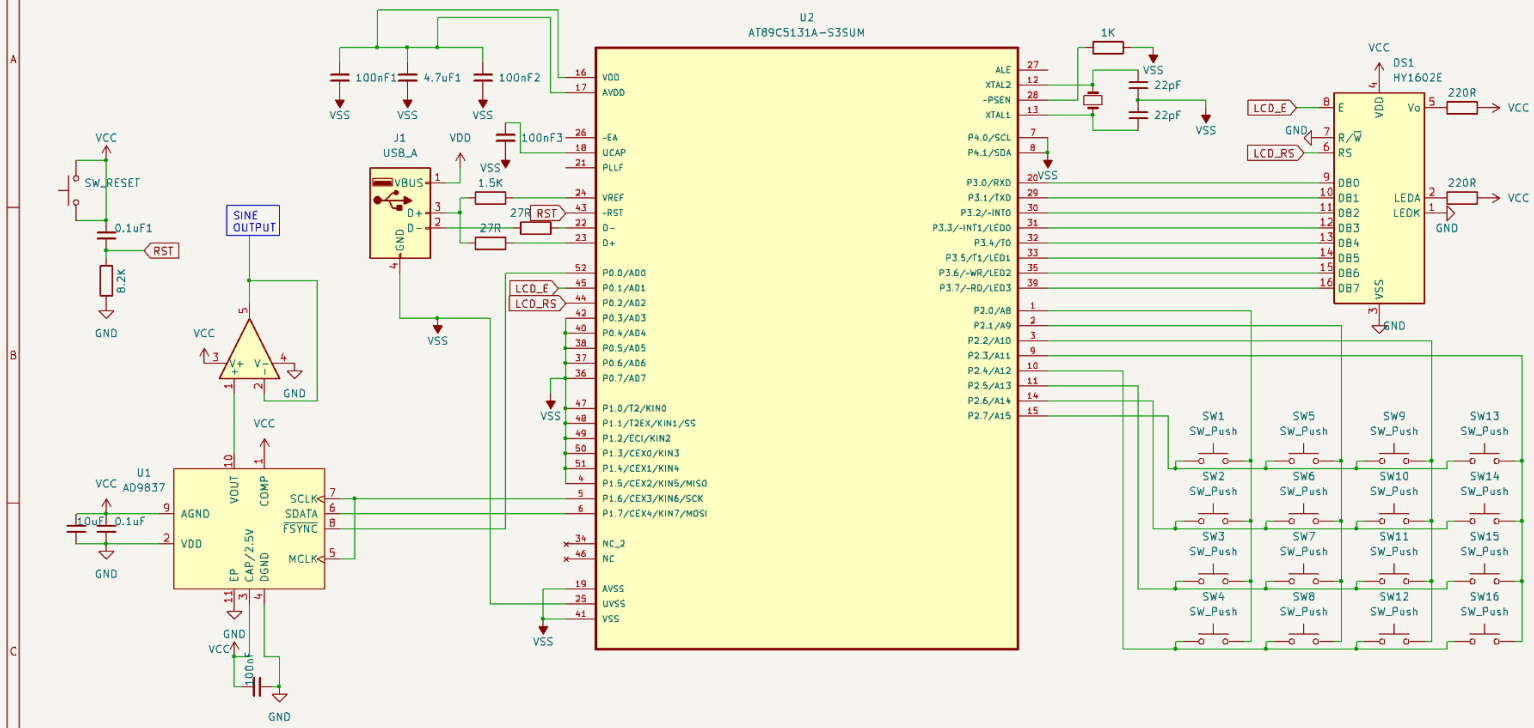
There are several details that have been left out, such as the means of powering the system. The device could be powered from USB, or from an internal battery so that it can work on its own, without a connection to a computer or power supply.

COSTS

AT89C5131A-S3SUM Microcontroller	**11.7 USD**
HY1602E LCD Display	**3.31 USD**
AD9837 Direct digital synthesis chip	**5.97 USD**
Membrane matrix keypad	**1.5 USD**
Total	**22.48 USD**

Prices shown as of 18/01/24, for 1 unit orders. Cost of manufacturing the PCB, as well as components such as resistors and capacitors is not included. Prices can be lowered when ordering in large quantities.

FULL SCHEMATIC



Daniel de Lizaur García

Sheet: /

File: emisy_project.kicad_sch

Title: Sine wave generator

Size: A4	Date:	**Rev:**
KiCad E.D.A. kicad 7.0.10		Id: 1/1